Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 4

Services Conducteurs & Maintenance, Bruno, Datafaker

Étudiants :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
4 / 7

4 avril 2026

Table des matières

1 Introduction **2**
1.1 Objectifs . . . 2

2 Architecture de persistance **3**
2.1 Bases de données . . . 3
2.2 Seeding automatique avec Datafaker . . . 3

3 Microservice Driver **5**
3.1 Modèle de données . . . 5
3.2 Surveillance des permis (LicenseScheduler) . . . 5
3.3 Endpoints REST . . . 5

4 Microservice Maintenance **6**
4.1 Modèle de données . . . 6
4.2 Détection des retards . . . 6
4.3 Endpoints REST . . . 6

5 Gateway GraphQL **7**
5.1 Architecture . . . 7
5.2 Agrégation multi-services . . . 7

6 Architecture événementielle (Saga) **8**
6.1 Format d'enveloppe (KafkaEventEnvelope) . . . 8
6.2 Transactions compensatoires . . . 8
6.3 Topics Kafka . . . 8

7 Authentification Keycloak **9**
7.1 Configuration . . . 9
7.2 Flux d'authentification . . . 9

8 Tests API avec Bruno **10**
8.1 Organisation de la collection . . . 10
8.2 Gestion automatique du token . . . 10
8.3 Environnements . . . 10
8.4 Bilan des tests REST . . . 11
8.5 Bilan des tests GraphQL . . . 11

9 Tests automatisés et couverture **12**
9.1 Stratégie . . . 12
9.2 Résultats JaCoCo . . . 12

10 Sécurisation et Maintenance (Hardening) 14
10.1 Remédiation applicative (Java) 14
10.2 Sécurisation des images Docker 14
10.3 Gestion des dépendances Node.js (Gateway) 14
10.4 Hygiène du dépôt Git 14

1. Introduction

Après la mise en place de l'infrastructure (S2) et du microservice Véhicules (S3), la semaine 4 complète la plateforme avec les services **Conducteurs** et **Maintenance**, la communication événementielle Kafka, la passerelle GraphQL et l'authentification Keycloak.

1.1. Objectifs

— **Service Conducteurs** — CRUD, suivi des permis, assignations véhicule–conducteur.

— **Service Maintenance** — Planification, historique par véhicule, alertes préventives.

— **Kafka** — Saga par chorégraphie pour les transitions d'état inter-services.

— **Gateway GraphQL** — Agrégation multi-services (Apollo Server).

— **Keycloak** — JWT + rôles pour sécuriser REST et GraphQL.

— **Bruno** — Collection de tests API (remplacement de Postman).

— **Datafaker** — Génération automatique de données de test au démarrage.

2. Architecture de persistance

> **Approche :** Une instance PostgreSQL unique (Bitnami Helm chart) héberge des bases séparées par service. L'isolation logique est assurée au niveau base de données, pas au niveau instance.

2.1. Bases de données

Les quatre bases sont créées automatiquement au démarrage de PostgreSQL via un script `initdb` déclaré dans les values Helm :

```sql
CREATE DATABASE driver_db;
CREATE DATABASE maintenance_db;
CREATE DATABASE vehicle_db;
CREATE DATABASE keycloak_db;
```

Listing 1 – Script d'initialisation (values.yaml fleet-infra)

- `vehicle_db`, `driver_db`, `maintenance_db` — une base par microservice métier.
- `keycloak_db` — créée par le script d'init Helm ; utilisée par Keycloak en déploiement Kubernetes.

Docker Compose : Keycloak se connecte à la base par défaut `postgres` (même instance PostgreSQL, pas de séparation `keycloak_db` dans ce profil). **Kubernetes :** Keycloak utilise `keycloak_db` (voir `infra/helm/fleet-infra` et le déploiement Keycloak).

Le schéma (DDL) est géré par Hibernate (`ddl-auto: update`) : pas de scripts SQL manuels à maintenir pour les entités JPA.

2.2. Seeding automatique avec Datafaker

Plutôt que des fichiers `data.sql` statiques, chaque service embarque un **DataSeeder** activé par le profil Spring `dev`. Le **vehicle-service** insère d'abord un véhicule **démo** à UUID fixe (aligné sur les requêtes Bruno/API) via un `INSERT` SQL natif — nécessaire car un UUID assigné à la main ne se comporte pas comme une entité JPA `new` avec `@GeneratedValue`. Ensuite, si la base ne contient qu'au plus ce véhicule, neuf véhicules supplémentaires sont générés avec **Datafaker** ; si la base est déjà peuplée d'autres lignes, le seed aléatoire est ignoré.

```java
@Transactional
public void run(String... args) {
  if (!vehicleRepository.existsById(DEMO_VEHICLE_ID)) {
    entityManager.createNativeQuery(
        "INSERT INTO vehicles (...) VALUES (?,?,...)").executeUpdate
    ();
  }
  if (vehicleRepository.count() > 1) return;
  for (int i = 0; i < 9; i++) {
    Vehicle v = new Vehicle();
    // champs remplis via Faker (plaque, VIN, etc.)
```

```
    vehicleRepository.save(v);
  }
}
```

Listing 2 – Logique résumée du DataSeeder (vehicle-service)

Service	Données générées	Détails
vehicle-service	10 véhicules (1 démo + 9 aléatoires)	Démo UUID fixe ; puis Faker si la base n'est
driver-service	8 conducteurs	Noms, emails, téléphones + 1 permis chacun
maintenance-service	12 maintenances	Types, priorités, dates, coûts pour les complé

Table 1 – Données générées par Datafaker (profil `dev`)

Le profil `dev` est activé via `SPRING_PROFILES_ACTIVE=dev` dans le template Helm (Kubernetes) et dans le `compose.yaml` (Docker). Les seeders **driver** et **maintenance** ne remplissent que des tables vides ; le seed **véhicule** garantit en outre la présence du véhicule démo si son UUID manque (bases déjà peuplées).

3. Microservice Driver

Objectif : Gérer le cycle de vie des conducteurs : profils, permis de conduire, coordination avec le service Véhicules lors des assignations.

3.1. Modèle de données

Relation **1-N** entre `Driver` et `DriverLicense`. Le service implémente le *Soft Delete* via `@SQLDelete` / `@SQLRestriction` Hibernate.

```java
@Entity @Table(name = "drivers")
@SQLDelete(sql = "UPDATE drivers SET deleted_at = NOW() WHERE id = ?"
    )
@SQLRestriction("deleted_at IS NULL")
public class Driver {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    private String firstName, lastName, email, phone, employeeId;

    @Enumerated(EnumType.STRING)
    private DriverStatus status = DriverStatus.ACTIVE;

    @OneToMany(mappedBy = "driver", cascade = CascadeType.ALL)
    private List<DriverLicense> licenses = new ArrayList<>();
}
```

Listing 3 – Entité Driver (extraits)

3.2. Surveillance des permis (LicenseScheduler)

Une tâche `@Scheduled` (cron quotidien) détecte les permis expirant sous 30 jours et publie des événements Kafka (`LICENSE_EXPIRING`, `LICENSE_EXPIRED`).

3.3. Endpoints REST

Méthode	Endpoint	Description
GET	`/api/drivers`	Liste (filtrable par statut)
POST	`/api/drivers`	Création d'un profil
GET	`/api/drivers/{id}`	Détails par UUID
PUT	`/api/drivers/{id}`	Mise à jour complète
PATCH	`/api/drivers/{id}/status`	Changement de statut
POST	`/api/drivers/{id}/licenses`	Ajout d'un permis
GET	`/api/drivers/{id}/licenses`	Liste des permis
DELETE	`/api/drivers/{id}`	Suppression logique

Table 2 – Endpoints REST — service Driver

4. Microservice Maintenance

Objectif : Traçabilité complète des interventions techniques, de la planification à la clôture, avec détection automatique des retards.

4.1. Modèle de données

L'entité `MaintenanceRecord` utilise trois énumérations (`MaintenanceType`, `MaintenanceStatus`, `MaintenancePriority`). Le lien vers le véhicule est un simple UUID — pas de clé étrangère inter-bases (principe microservices).

4.2. Détection des retards

Une tâche `@Scheduled` quotidienne (01h00) passe les maintenances en retard au statut `OVERDUE` et publie une alerte Kafka.

4.3. Endpoints REST

Méthode	Endpoint	Description
GET	`/api/maintenance`	Liste (filtrable par véhicule, statut, priorité)
POST	`/api/maintenance`	Planification d'une intervention
GET	`/api/maintenance/vehicle/{id}`	Historique d'un véhicule
PUT	`/api/maintenance/{id}`	Mise à jour générale
PATCH	`/api/maintenance/{id}/status`	Clôture (coût, km, notes)

Table 3 – Endpoints REST — service Maintenance

5. Gateway GraphQL

> **Objectif :** Agréger les trois services métier derrière une API GraphQL unifiée (Apollo Server, Node.js).

5.1. Architecture

Le Gateway est déployé sur le port 4000 et route les requêtes via des clients REST (Axios) vers chaque service backend. Le JWT reçu du client est **forwardé** automatiquement aux services.

- **VehicleClient** → `http://vehicle-service:8080`
- **DriverClient** → `http://driver-service:8080`
- **MaintenanceClient** → `http://maintenance-service:8080`

Un mécanisme de retry (intercepteur Axios) gère les erreurs réseau transitoires au démarrage (`ECONNREFUSED`, `ECONNRESET`).

5.2. Agrégation multi-services

Les relations suivantes sont résolubles en une seule requête GraphQL :

- **Véhicule** ↔ **Maintenance** — historique technique depuis la fiche véhicule.
- **Véhicule** ↔ **Conducteur** — assignation courante et profil du conducteur.
- **Maintenance** ↔ **Conducteur** — technicien assigné à une intervention.

6. Architecture événementielle (Saga)

> **Principe :** Saga par **chorégraphie** — chaque service réagit aux événements des autres et publie ses propres résultats. Pas d'orchestrateur central.

6.1. Format d'enveloppe (KafkaEventEnvelope)

```
{
  "eventId": "625c87b9-...",
  "eventType": "MAINTENANCE_COMPLETED",
  "timestamp": 1775220800.293,
  "payload": { "vehicle_id": "39e75ab3-...", "cost_eur": 250.50 },
  "metadata": { "triggered_by": "maintenance-service" }
}
```

Listing 4 – Enveloppe événement unifiée

6.2. Transactions compensatoires

1. **T1** — Maintenance publie `MAINTENANCE_STARTED`.
2. **T2** — Véhicule consomme et passe en `IN_MAINTENANCE`.
3. **C1** — En cas d'échec, Véhicule publie `MAINTENANCE_REJECTED` → Maintenance annule la fiche.

6.3. Topics Kafka

Topic	Événements	Effet
`flotte.maintenance.events`	`STARTED`, `COMPLETED`, `CANCELLED`	Véhicule : `available`
`flotte.conducteurs.events`	`DRIVER_CREATED/UPDATED`, `LICENSE_EXPIRING/EXPIRED`	Notification int
`flotte.assignments.events`	`VEHICLE_ASSIGNED/UNASSIGNED`	Driver : `ON_TOU`

Table 4 – Topics Kafka et flux d'événements

7. Authentification Keycloak

> **Approche :** Keycloak (v26.5.7) déployé avec `-import-realm` pour charger automatiquement le realm `gestion-flotte`, ses clients et ses utilisateurs au démarrage.

7.1. Configuration

- **Base PostgreSQL** — sous **Kubernetes**, Keycloak utilise `keycloak_db`. Sous **Docker Compose**, la connexion JDBC pointe sur la base `postgres` (voir variables `KC_DB_URL`).
- **Realm** — `gestion-flotte`, importé depuis `keycloak/realm-export.json`.
- **Client** — `admin-cli` (public, `directAccessGrantsEnabled`).
- **Utilisateur** — `test-user` (rôle `admin`), credentials hashées (argon2) dans l'export.

7.2. Flux d'authentification

Les services Spring Boot valident les JWT via le `jwk-set-uri` de Keycloak. Le Gateway GraphQL forward le header `Authorization` reçu du client vers chaque service backend.

```
POST /auth/realms/gestion-flotte/protocol/openid-connect/token
  grant_type=password
  client_id=admin-cli
  username=test-user
  password=password
```

Listing 5 – Récupération du token (grant type password)

8. Tests API avec Bruno

> **Migration Postman → Bruno :** la collection est désormais au format Bruno (fichiers `.bru` versionnés dans Git), avec gestion automatique du token JWT via un script de collection.

8.1. Organisation de la collection

```
bruno/
  bruno.json                  # Metadonnees collection
  collection.bru              # Script pre-request : token JWT + en-
    tete Authorization
  environments/
    Minikube.json             # Variables flotte.local (K8s)
    Docker.json               # Variables localhost (Compose)
  REST/
    service-vehicule/         # CRUD vehicules
    service-driver/           # CRUD conducteurs
    service-maintenance/      # CRUD maintenance
  GraphQL/
    service-vehicule/         # Query vehicles
    service-driver/           # Query drivers
    service-maintenance/      # Query maintenanceRecords
```

Listing 6 – Structure du dossier bruno/

8.2. Gestion automatique du token

Le fichier `collection.bru` définit `auth: none` et un script `pre-request` qui s'exécute avant chaque requête. Le renouvellement du JWT utilise **`bru.sendRequest`** (API Bruno ; `fetch` et `URLSearchParams` ne sont pas toujours disponibles dans le sandbox). Le corps `application/x-www-form-urlencoded` est construit manuellement ; la date d'expiration est lue dans le payload du JWT. Les variables d'environnement sont lues via `bru.getEnvVar`, avec secours `bru.getAllEnvVars()` et valeurs par défaut alignées sur `Docker.json` lorsque Bruno ne résout pas l'environnement dans le script de collection. Aucune requête POST séparée pour obtenir le jeton n'est nécessaire.

8.3. Environnements

Deux environnements sont fournis, sélectionnables dans Bruno :

- **Minikube** — `base_url = http://flotte.local` (Ingress K8s).
- **Docker** — `base_url = http://127.0.0.1:4000` (gateway), `keycloak_token_url` sur le port 8180 avec préfixe `/auth` (Keycloak en Compose).

8.4. Bilan des tests REST

Service	Endpoint	Code attendu	Résultat
Driver	POST /api/drivers	201	**OK**
Driver	GET /api/drivers	200	**OK**
Driver	GET /api/drivers/{id}	200	**OK**
Driver	PUT /api/drivers/{id}	200	**OK**
Driver	PATCH /api/drivers/{id}/status	200	**OK**
Driver	POST /api/drivers/{id}/licenses	201	**OK**
Driver	GET /api/drivers/{id}/licenses	200	**OK**
Driver	DELETE /api/drivers/{id}	204	**OK**
Maintenance	POST /api/maintenance	201	**OK**
Maintenance	GET /api/maintenance	200	**OK**
Maintenance	GET /api/maintenance/vehicle/{id}	200	**OK**
Maintenance	PUT /api/maintenance/{id}	200	**OK**
Maintenance	PATCH /api/maintenance/{id}/status	200	**OK**
Vehicle	POST /api/vehicles	201	**OK**
Vehicle	GET /api/vehicles	200	**OK**
Vehicle	PUT /api/vehicles/{id}	200	**OK**
Vehicle	POST /api/vehicles/{id}/assignments	201	**OK**
Vehicle	GET /api/vehicles/{id}/assignments	200	**OK**

Table 5 – Bilan des 18 endpoints REST testés via Bruno

8.5. Bilan des tests GraphQL

Domaine	Opération	HTTP	Résultat
Véhicules	Query `vehicles`	200	**OK**
Maintenance	Query `maintenanceRecords`	200	**OK**
Conducteurs	Query `drivers`	200	**OK**

Table 6 – Bilan des requêtes GraphQL (Gateway, JWT via Bruno)

9. Tests automatisés et couverture

> **Objectif :** Couverture minimale de **80% en lignes** sur les tests unitaires (JaCoCo).

9.1. Stratégie

— **Tests unitaires** (JUnit 5, Mockito, `@WebMvcTest`) — validation des services et endpoints.

— **Tests d'intégration** (`@SpringBootTest`, H2) — flux HTTP + persistance.

Le profil Maven `unit-coverage` isole les tests unitaires pour le rapport JaCoCo :

```
./mvnw clean test -Punit-coverage
# Rapport : target/site/jacoco/index.html
```

Listing 7 – Génération du rapport JaCoCo

9.2. Résultats JaCoCo

driver-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.driver.config		0%		0%	12	12	22	22	10	10	1	1
com.flotte.driver.controllers		55%		n/a	4	9	4	12	4	9	0	1
com.flotte.driver.models		88%		n/a	4	40	6	54	4	40	0	2
com.flotte.driver		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.driver.events		98%		n/a	1	13	1	23	1	13	0	7
com.flotte.driver.services		100%		75%	4	23	0	103	0	15	0	1
com.flotte.driver.events.producers		100%		n/a	0	5	0	37	0	5	0	1
com.flotte.driver.dto		100%		n/a	0	6	0	6	0	6	0	6
com.flotte.driver.events.consumers		100%		100%	0	6	0	22	0	4	0	2
com.flotte.driver.models.enums		100%		n/a	0	2	0	13	0	2	0	2
com.flotte.driver.jobs		100%		n/a	0	5	0	17	0	5	0	1
Total	151 of 1,392	89%	8 of 24	66%	26	123	35	312	20	111	1	25

FIGURE 1 – JaCoCo — `driver-service`

maintenance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.maintenance.config		0%		0%	11	11	19	19	9	9	1	1
com.flotte.maintenance.service		91%		62%	23	47	4	83	1	18	0	1
com.flotte.maintenance.events		93%		60%	7	30	3	56	3	25	1	7
com.flotte.maintenance.model		90%		n/a	5	36	5	54	5	36	0	4
com.flotte.maintenance.controller		85%		n/a	1	7	1	9	1	7	0	1
com.flotte.maintenance		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		100%		n/a	0	4	0	4	0	4	0	4
com.flotte.maintenance.dto		100%		n/a	0	3	0	3	0	3	0	3
Total	177 of 1,215	85%	30 of 72	58%	48	140	34	231	20	104	2	22

FIGURE 2 – JaCoCo — `maintenance-service`

vehicle-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.vehicle.config		0%		0%	12	12	23	23	10	10	1	1
com.flotte.vehicle.services		88%		58%	12	26	2	127	7	20	0	1
com.flotte.vehicle.models		88%		n/a	5	45	5	52	5	45	0	2
com.flotte.vehicle.events.consumers		88%		58%	4	13	3	35	0	5	0	1
com.flotte.vehicle		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		98%		n/a	1	14	1	23	1	14	0	6
com.flotte.vehicle.dto		100%		n/a	0	7	0	15	0	7	0	6
com.flotte.vehicle.events.producers		100%		n/a	0	5	0	16	0	5	0	1
com.flotte.vehicle.controllers		100%		n/a	0	10	0	13	0	10	0	1
com.flotte.vehicle.models.enums		100%		n/a	0	2	0	10	0	2	0	2
com.flotte.maintenance.model		100%		n/a	0	1	0	6	0	1	0	1
com.flotte.maintenance.events		100%		n/a	0	1	0	1	0	1	0	1
Total	202 of 1,509	86%	14 of 28	50%	35	138	36	324	24	122	1	24

FIGURE 3 – JaCoCo — `vehicle-service`

Service	Couverture lignes	Objectif
`driver-service`	$\approx 89\%$	$\geq 80\%$
`maintenance-service`	$\approx 85\%$	$\geq 80\%$
`vehicle-service`	$\approx 89\%$	$\geq 80\%$

TABLE 7 – Synthèse couverture JaCoCo (profil `unit-coverage`)

10. Sécurisation et Maintenance (Hardening)

> **Audit de sécurité :** L'utilisation de l'outil **Trivy** dans la pipeline CI/CD a permis d'identifier et de corriger plusieurs vulnérabilités critiques dans les images Docker et les dépendances logicielles.

10.1. Remédiation applicative (Java)

Les services **Driver**, **Maintenance** et **Vehicle** ont été mis à jour pour corriger des failles dans les composants embarqués (notamment Tomcat et Spring Security) :

- **Spring Boot** — Parent Maven `spring-boot-starter-parent` en `3.4.0` (services Java) ; les dépendances transitives (Tomcat, Spring Security) suivent le BOM du parent.
- **PostgreSQL Driver** — Version gouvernée par le BOM Spring Boot (surclassement possible selon les audits).
- **Dependency Management** — Lors des audits de sécurité, surcharges ponctuelles des versions de `tomcat-embed-core` et `spring-security` pour traiter les CVE identifiés (ex. CVE-2025-24813, CVE-2026-22732 selon les rapports d'audit).

10.2. Sécurisation des images Docker

Pour réduire la surface d'attaque des conteneurs :

- **Update OS** — Ajout systématique de `apk upgrade -no-cache` dans les Dockerfiles pour appliquer les patchs de sécurité Alpine Linux au moment du build.
- **Images de base** — Utilisation de `maven:3.9.9` pour la construction et `eclipse-temurin:21-jre` pour l'exécution.

10.3. Gestion des dépendances Node.js (Gateway)

Le Gateway GraphQL a fait l'objet d'un durcissement particulier suite à la détection de failles transitives dans les outils de package (`npm`) :

- **npm Overrides** — Utilisation de la directive `overrides` dans le `package.json` pour forcer la version `4.0.4` de la bibliothèque `picomatch` (résolution de la CVE-2026-33671).
- **npm Audit** — Exécution de `npm audit fix` pour assainir l'arbre des dépendances.

10.4. Hygiène du dépôt Git

Afin d'éviter la pollution du serveur et les problèmes de compatibilité, le fichier `.gitignore` a été complètement revu. Les dossiers `node_modules/`, `target/` et `dist/` ont été définitivement retirés de l'historique Git via une purge de l'index (`git rm -cached`).